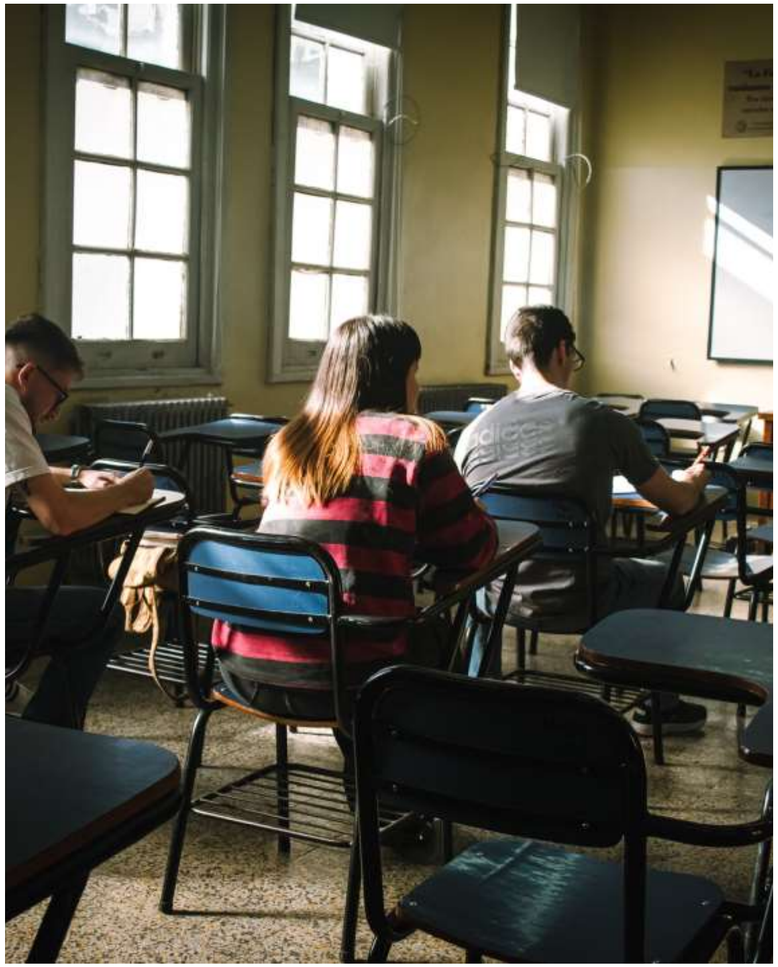
Foundations of Artificial Intelligence

AI-Based Exam Duty and Laboratory Allocation System

Using constraint satisfaction and search algorithms



Presented by **Karthikeyan A**
Kaarthika S

Allocate once, repair on demand

Exam cells assign venues and invigilators for hundreds of sessions by hand, and every lab failure, postponement or faculty leave forces a manual redo that disturbs assignments unrelated to the change.

We model the task as a Constraint Satisfaction Problem and generate the allocation with AC-3 propagation and heuristic backtracking search. Incidents are repaired with min-conflicts local search under a change penalty, so only the affected slots move. Infeasible cases are explained in terms of the blocking constraint.

- CSP + backtracking (MRV, LCV, forward checking)
- Min-conflicts repair with minimal disruption
- Validator and infeasibility explanation



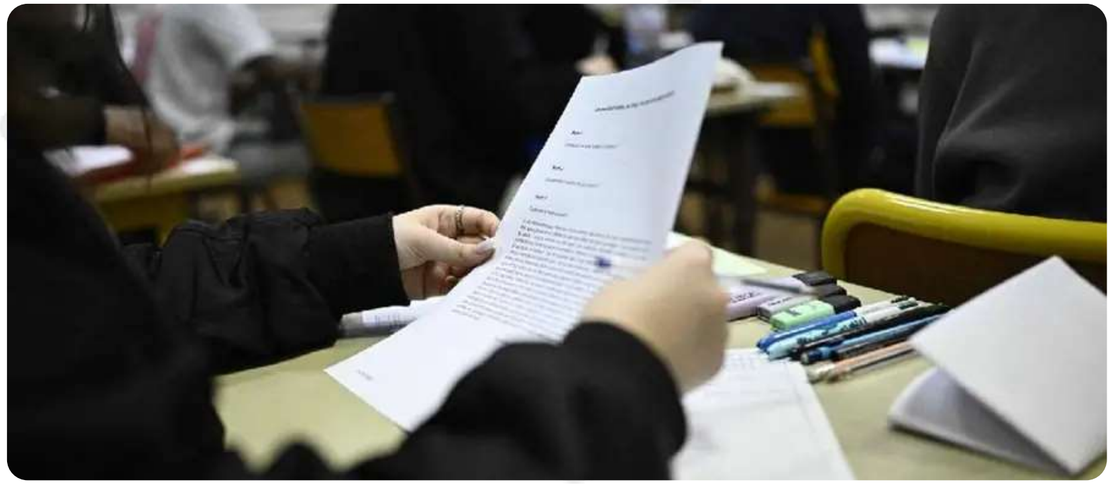
How exam allocation is done today, and where it breaks

Manual allocation

Venues and invigilators are assigned in spreadsheets. Clashes with lectures, over-capacity rooms and labs with too few working systems are caught late or not at all.

Fragile to change

One broken lab or postponed exam means re-checking the entire sheet, and faculty learn of changes hours before duty.



Given the exam timetable, rooms and faculty schedules, assign every exam a venue and invigilators so that all institutional rules hold, and when an incident occurs, repair the allocation with the fewest possible changes, or explain why no repair exists.

What has been done

Examination timetabling is a standard CSP benchmark. Surveys by Qu et al. (2009) and Siew et al. (2024) cover graph colouring, constraint methods and metaheuristics; all target generation from scratch.



Constraint foundations

Mackworth (1977) introduced AC-3; Russell & Norvig define MRV, LCV, forward checking and min-conflicts.

Exam timetabling surveys

Qu et al. (2009), Siew et al. (2024): assign exams to periods and rooms; invigilators treated as post-processing.

Repair methods

Minton et al. (1992) showed min-conflicts repairs near-correct schedules in few steps; rarely applied to invigilator duties.

Where existing systems stop

Timetabling research has solved the generation problem well: given exams, periods and rooms, surveys. Yet three needs of a working examination cell remain unaddressed.

The schedule is treated as disposable and regenerated after every change, invigilator duties are left outside the model, and infeasible cases are reported as plain failure. This project targets all three with a single CSP model solved by two complementary search algorithms.

Existing 1

Any change triggers a full re-solve, so exams and staff unrelated to the incident are moved too. Faculty get a new duty chart for the week because one lab failed.

Ours 1

Only affected assignments are released and repaired with min-conflicts under a change penalty λ. A broken lab moves one exam; only faculty whose duties changed are notified.

Existing 2

Rooms and periods are modelled, but faculty timetables, leave and duty limits are not, so invigilator allocation remains a manual step afterwards.

Ours 2

Venue and invigilators are assigned together as one CSP under H1–H7, so no faculty member is placed in a hall during their own lecture or beyond the daily duty limit.

Objectives

Formulate

Model allocation as a CSP with hard constraints H1–H7 and weighted soft constraints S1–S5

Generate

Implement AC-3 and backtracking with MRV, degree, LCV and forward checking

Repair

Implement min-conflicts with a change penalty λ for incident-driven rescheduling

Validate and deliver

Independent validator, infeasibility explainer, role-based web app with college login



Generate, repair, explain

The published schedule is treated as a commitment. A complete search algorithm builds it from nothing; a fast local search protects it when something breaks.

01

Generate

AC-3 prunes domains, backtracking assigns venue and invigilators per exam, hill climbing balances duty load

02

Repair

Incident releases only affected slots; min-conflicts reassigns with penalty λ per change; validator re-checks

03

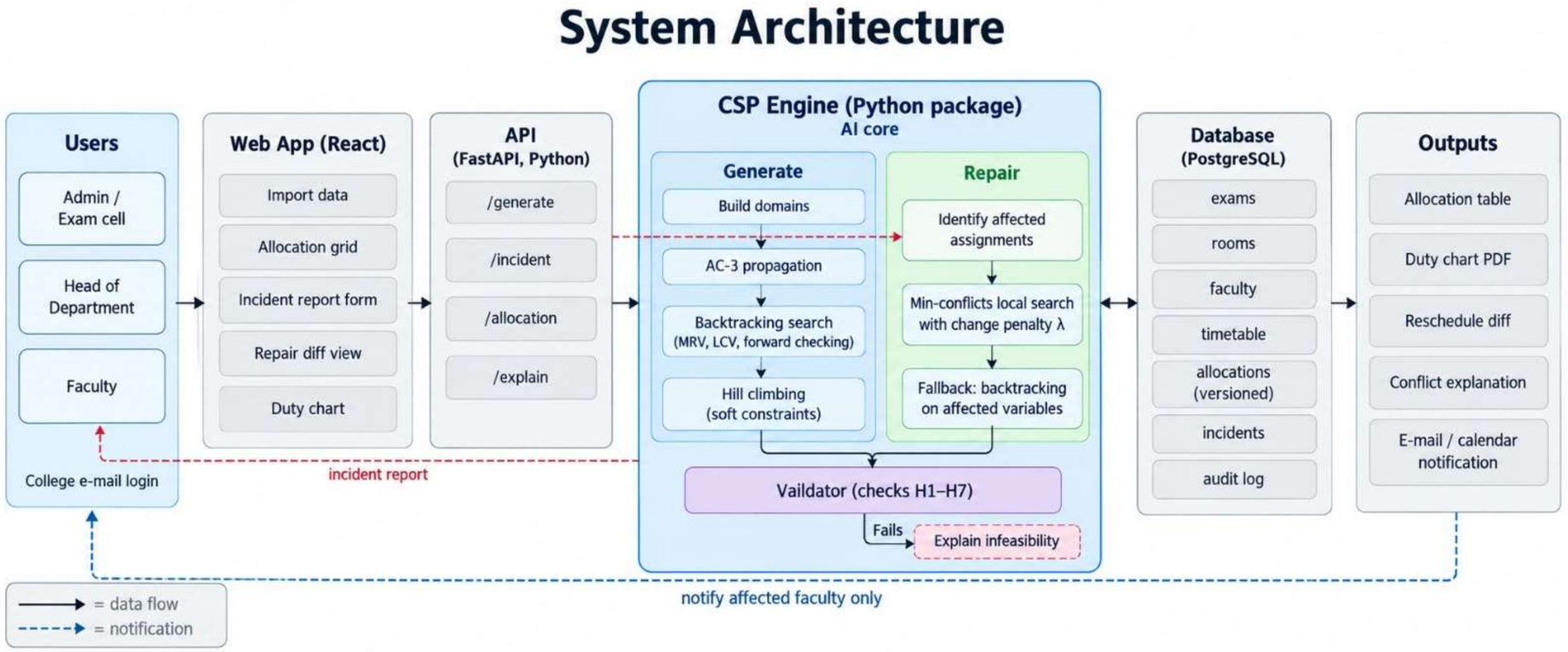
Explain

If no repair exists, name the blocking constraint and suggest a relaxation: new date, larger room, one extra duty

Architecture

Administrator imports exams, rooms, faculty and timetable; the CSP engine builds domains, runs AC-3, backtracking and hill climbing; results are validated and stored as versions.

Incident reports enter the rescheduling module, which repairs with min-conflicts and publishes a diff to affected faculty only.



Four algorithms, implemented from scratch

Four textbook algorithms, each written in Python without a solver library. Backtracking is complete and builds the schedule from nothing; min-conflicts starts from the published schedule and converges in a few steps, which is exactly the repair case

AC-3

Removes room/staff values with no support in neighbouring exams; detects infeasibility before search

Backtracking + heuristics

MRV picks the most constrained exam, degree breaks ties, LCV orders rooms, forward checking prunes after each assignment

Min-conflicts repair

Starts from the published schedule; change penalty λ keeps unaffected assignments fixed; falls back to backtracking on affected variables

Hill climbing

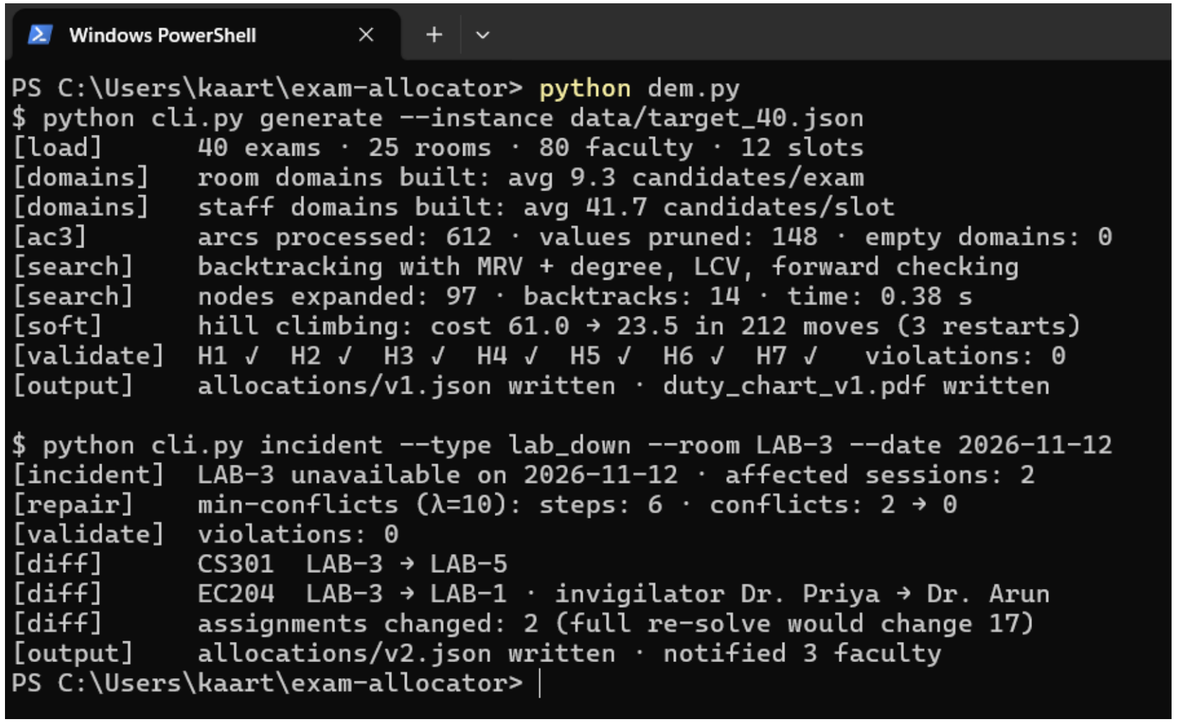
Swaps assignments to lower soft cost (fairness, proximity) while keeping hard feasibility.

Pseudo Code:

```
BACKTRACK(A): X ← MRV variable; for v in LCV order:
  if consistent(H1..H7): assign, forward-check, recurse; undo
MIN-CONFLICTS(published, incident): release affected vars;
  repeat: X ← conflicted var; X ← argmin conflicts + λ·[changed]
```

Stack and status

The CSP engine is a standalone Python package (ac3, backtracking, local_search, cost, validator, explain) exercised from the command line before any UI exists.



Backend

Python 3.12, FastAPI, PostgreSQL with versioned allocations

Frontend and auth

React, college-domain login (roster now, Google OAuth later), e-mail and ICS notifications

Data and tests

Synthetic generator (40 exams, 25 rooms, 80 faculty, 12 slots); pytest, scenario tests, validator property tests

Outputs and evaluation

Exam duty allocation — Nov 2026 · Version 2

Date	Slot	Course	Strength	Venue	Invigilator(s)	Status
10 Nov	FN	CS301 Operating Systems	58	LAB-5	Dr. Priya, Mr. Karthik	OK
10 Nov	AN	EC204 Digital Circuits	62	LAB-1	Dr. Arun, Ms. Divya, Mr. Rahul	OK
10 Nov	AN	MA201 Linear Algebra	115	HALL-A	Dr. Meena, Dr. Suresh, Ms. Anitha, Mr. Vijay	OK
11 Nov	FN	CS305 Databases	55	LAB-2	Dr. Lakshmi, Mr. Ganesh	OK
11 Nov	FN	EE210 Signals	88	HALL-B	Dr. Kumar, Ms. Shalini, Mr. Mohan	OK
12 Nov	FN	CS310 Networks	60	LAB-3 → LAB-5	Dr. Priya → Dr. Arun, Mr. Karthik	CHANGED
12 Nov	AN	IT202 Web Tech	45	LAB-4	Ms. Divya, Mr. Rahul	OK

Reschedule diff · incident #14

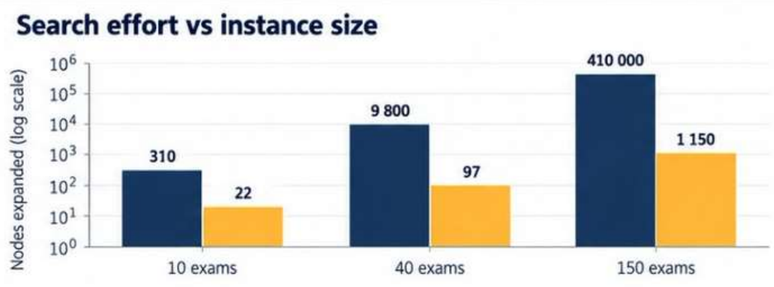
LAB-3 unavailable on 12 Nov.

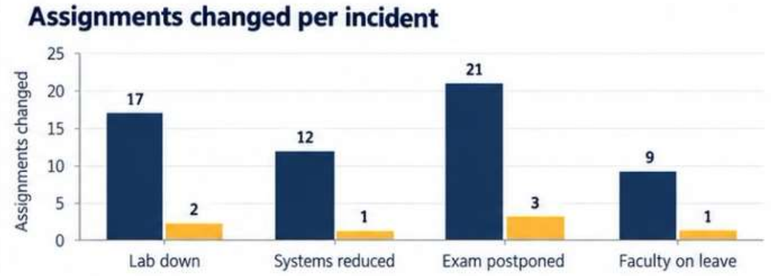
Repaired with min-conflicts in 6 steps.

Assignments changed: 2 (full re-solve: 17).

Validator: 0 violations.

Notify 3 affected faculty





Plain backtracking | MRV + LCV + forward checking | Full re-solve | Min-conflicts repair

Synthetic instances · mean of 10 runs · validator reported 0 hard-constraint violations in all cases

Allocation grid, duty chart PDF, reschedule diff ("CS301: Lab-2 → Lab-5; Dr A → Dr B") and conflict explanation. Evaluation on synthetic instances with planned experiments: heuristic ablation, repair vs full re-solve, fairness.

- 0 hard-constraint violations on every published allocation
- ≤ 5 assignments changed per incident (vs full re-solve)
- < 2 s repair, < 5 s generation on the target instance